

Практика 2. Модель та навчання в 2D бінарному дискретному середовищі

Курс: Вступ до загального штучного інтелекту

Осауленко Вячеслав

2026-01-01

Зміст

1	Теоретичне підґрунтя	2
1.1	Середовище	2
1.1.1	Топологія	2
1.1.2	Складність	3
1.2	Агент	3
1.3	Сприйняття	3
1.4	Модель світу: від растрової карти до об'єктної	3
1.4.1	Підхід 1 — Растрова карта (реалізовано в коді)	4
1.4.2	Підхід 2 — Об'єктна модель (завдання цієї практики)	4
1.4.3	Об'єкти в Game of Life	4
1.4.4	Порівняння ознак: фазово-інваріантна відстань	5
1.4.5	Групування за ознаками	5
2	Архітектура програми	5
2.1	Клас World	6
2.2	Клас Agent	6
2.3	Клас SaliencyMap	6
2.4	Клас WorldModel	7
2.5	Клас ObjectTracker та WorldObject	7
2.6	Клас ObjectHistory	8
2.7	Політики руху (action.py)	8
2.8	Клас PatternMatcher	9
2.9	Візуалізація (viz_run.py)	9
3	Структура коду	9
3.1	Точка входу	9
3.2	Головний цикл (simulation.py)	10
3.3	Формат патернів	10
4	Завдання	10
4.1	Завдання 1 — Запуск та ознайомлення	10
4.2	Завдання 2 — Об'єктна модель світу	11
4.2.1	2.1. Групування об'єктів за ознаками	11
4.2.2	2.2. Побудова об'єктної моделі	11
4.3	Завдання 3 — Реконструкція та порівняння з реальністю	12
4.3.1	3.1. Реконструкція растрової карти з об'єктної моделі	12

4.3.2	3.2. Похибка реконструкції (відстань Геммінга)	12
4.3.3	3.3. Порівняння розмірів моделей	13
4.4	Завдання 4 (Не обов'язкове) — Динамічні об'єкти	14
4.4.1	4.1. Передбачення для осциляторів	14
4.4.2	4.2. Вимірювання точності передбачення	15
4.5	Завдання 5 — Аналіз та висновки	15
5	Критерії оцінювання	15
6	Корисні матеріали	16

1 Теоретичне підґрунтя

1.1 Середовище

У Практиці 1 агент діяв у **1D статичному** середовищі. Тепер переходимо до значно складнішого випадку — **2D бінарне дискретне динамічне** середовище.

Середовище — це прямокутна сітка розміром $W \times H$ бітів:

$$\mathcal{W}_t = \{w_{x,y}^{(t)}\}_{x \in [0,W), y \in [0,H)}, \quad w_{x,y} \in \{0, 1\}$$

Ключова відмінність від Практики 1: середовище **динамічне** — воно змінюється кожен крок за правилами **клітинного автомата Конвея** (Game of Life):

- **Народження:** мертва клітина з рівно 3 живими сусідами стає живою
- **Виживання:** жива клітина з 2 або 3 живими сусідами залишається живою
- **Смерть:** у всіх інших випадках клітина стає мертвою

Формально:

$$w_{x,y}^{(t+1)} = \begin{cases} 1, & \text{якщо } n_{x,y}^{(t)} = 3 \\ w_{x,y}^{(t)}, & \text{якщо } n_{x,y}^{(t)} = 2 \\ 0, & \text{інакше} \end{cases}$$

де $n_{x,y}^{(t)} = \sum_{(i,j) \in \mathcal{N}(x,y)} w_{i,j}^{(t)}$ — кількість живих сусідів у 8-зв'язковому оточенні (Мура).

1.1.1 Топологія

Середовище має **тороїдальну** топологію — протилежні краї з'єднані:

$$w_{x,y} = w_{x \bmod W, y \bmod H}$$

Це означає: агент, що вийшов за правий край, з'являється зліва; за нижній — зверху.

1.1.2 Складність

Кількість можливих станів: $|S| = 2^{W \times H}$. При $W = H = 50$ це $2^{2500} \approx 10^{752}$ — астрономічне число. Навіть при малому вікні сприйняття, простір спостережень $2^{(2S+1)^2}$ — при $S = 9$ маємо 2^{361} станів.

Game of Life породжує **складну динаміку** з простих правил. Серед відомих патернів:

Тип	Приклад	Поведінка
Still life	Block (2×2)	Незмінний: $w^{(t+1)} = w^{(t)}$
Oscillator	Blinker (період 2)	Циклічний: $w^{(t+k)} = w^{(t)}$
Spaceship	Glider	Цикл + зсув: рухається по сітці
Chaos	—	Непередбачувана еволюція

Це робить задачу побудови моделі світу принципово складнішою, ніж у статичному 1D випадку.

1.2 Агент

Агент характеризується:

- **Позицією** $(x_t, y_t) \in [0, W) \times [0, H)$
- **Дією** $a_t \in \{\text{up, down, left, right}\}$ — крок на 1 клітину

Дія змінює позицію:

$$x_{t+1} = (x_t + \Delta x) \bmod W, \quad y_{t+1} = (y_t + \Delta y) \bmod H$$

1.3 Сприйняття

Агент бачить квадратне вікно $(2S + 1) \times (2S + 1)$ навколо своєї позиції (з тороїдальним загортанням):

$$o_t = \{w_{(x_t+i) \bmod W, (y_t+j) \bmod H}\}_{i,j \in [-S, S]}$$

При $S = 9$ агент бачить патч $19 \times 19 = 361$ клітину — це лише $\approx 14\%$ від світу 50×50 .

1.4 Модель світу: від растрової карти до об'єктної

У Практиці 1 ми розглянули два підходи: словниковий (егоцентричний) та симуляційний (аллоцентричний). В 2D динамічному середовищі ситуація складніша.

1.4.1 Підхід 1 — Растрова карта (реалізовано в коді)

Агент зберігає **повну копію середовища** $\hat{W} \in \{0, 1, ?\}^{H \times W}$, де ? означає «ще не спостережено». Кожен крок агент «штампую» своє сприйняття в карту:

$$\hat{w}_{x,y}^{(t)} = \begin{cases} w_{x,y}^{(t)}, & \text{якщо } (x, y) \text{ потрапляє у вікно сприйняття} \\ \hat{w}_{x,y}^{(t-1)}, & \text{інакше (застаріле значення)} \end{cases}$$

Проблема: у динамічному середовищі карта **застаріває**. Клітини за межами поточного вікна продовжують еволюціонувати, але агент цього не знає. Карта точна лише в зоні поточного сприйняття.

Розмір: $W \times H$ бітів + $W \times H$ бітів маски $_known = 2 \cdot W \cdot H$ бітів.

1.4.2 Підхід 2 — Об'єктна модель (завдання цієї практики)

Замість зберігання кожної клітини, агент **виділяє об'єкти** — скупчення живих клітин — і описує світ як колекцію пар:

$$\mathcal{M}_t = \{(f_i, p_i)\}_{i=1}^K$$

де:

- f_i — **ознака** (feature) об'єкта: бінарний патерн його форми
- p_i — **позиція** об'єкта: координати центроїда у світових координатах

Це **стисне представлення**: замість $W \times H$ значень ми зберігаємо лише K об'єктів, де $K \ll W \cdot H$.

Аналогія з когнітивною наукою

Растрова карта — це «фотографічна пам'ять»: точна, але об'ємна.
Об'єктна модель — це **семантична пам'ять**: «тут блок, там блінкер, там глайдер». Людина не запам'ятовує кожен піксель сцени — вона запам'ятовує **об'єкти та їх розташування**.
Це фундаментальна ідея: ефективна модель світу — це модель, яка **стискає** реальність до компактного опису, зберігаючи лише суттєве.

1.4.3 Об'єкти в Game of Life

У Game of Life об'єкти мають природну класифікацію:

Статичні об'єкти (still life) мають період 1: $f_i^{(t+1)} = f_i^{(t)}$. Їх ознака — фіксований бінарний патерн.

Осцилятори мають період $k > 1$: об'єкт циклічно проходить через k різних станів:

$$f_i^{(t)} = f_i^{(t+k)}, \quad \text{cycle}_i = [f_i^{(0)}, f_i^{(1)}, \dots, f_i^{(k-1)}]$$

Ознакою осцилятора є його **цикл** — впорядкована послідовність з k бінарних кадрів. Два осцилятори вважаються однаковими, якщо їх цикли збігаються (з точністю до фази).

Рухомі об'єкти (spaceships) — осцилятори, що додатково зміщуються на $(\Delta x, \Delta y)$ за період. Їх позиція змінюється:

$$p_i^{(t+k)} = p_i^{(t)} + (\Delta x_i, \Delta y_i)$$

1.4.4 Порівняння ознак: фазово-інваріантна відстань

Для порівняння двох об'єктів з циклами потрібна міра подібності, **інваріантна до фази**. Якщо об'єкт А має цикл $[a_0, a_1, \dots, a_{k-1}]$, а об'єкт В має цикл $[b_0, b_1, \dots, b_{k-1}]$, то:

$$d(A, B) = \min_{\phi \in [0, k)} \sum_{j=0}^{k-1} \|a_j - b_{(j+\phi) \bmod k}\|_1$$

Тобто ми зсуваємо одну послідовність відносно іншої і беремо мінімальну відстань Гемінга.

Для об'єктів з різним періодом — вони автоматично вважаються різними (різний тип).

1.4.5 Групування за ознаками

Об'єкти з однаковими (або дуже схожими) ознаками утворюють **клас**. Об'єктна модель перетворюється на ще компактніше представлення:

$$\mathcal{M}_t = \{(F_c, \{p_{c,1}, p_{c,2}, \dots\})\}_{c=1}^C$$

де F_c — ознака класу (зберігається **один раз**), а $\{p_{c,j}\}$ — позиції всіх об'єктів цього класу. При $C \ll K$ це дає додаткове стиснення.

2 Архітектура програми

Код розбитий на модулі, кожен з яких відповідає за окрему частину системи. Залежності між модулями утворюють таку ієрархію:

2d_binary_discrete_ca_saliency.py	← точка входу
└─ simulation.py	← головний цикл + клавіатурне управління
└─ world.py	← середовище (Game of Life)
└─ agent.py	← агент (позиція, сприйняття, рух)
└─ saliency.py	← карти салієнтності (виділення значимих ділянок)
└─ model.py	← внутрішня модель світу (WorldModel)
└─ objects.py	← трекінг об'єктів (ObjectTracker, ObjectHistory)
└─ action.py	← політики автоматичного руху
└─ patterns.py	← завантаження та порівняння патернів
└─ viz_rerun.py	← візуалізація через Rerun

2.1 Клас World

```
class World:
    def __init__(self, width=20, height=20):
        self.state = np.random.randint(0, 2, size=(height, width)).astype(np.uint8)
        self.time_step = 0

    def step(self):
        p = np.pad(self.state, 1, mode='wrap')
        neighbours = (p[:-2, :-2] + p[:-2, 1:-1] + p[:-2, 2:] +
                      p[1:-1, :-2] + p[1:-1, 2:] +
                      p[2:, :-2] + p[2:, 1:-1] + p[2:, 2:])
        survive = (self.state == 1) & ((neighbours == 2) | (neighbours == 3))
        born = (self.state == 0) & (neighbours == 3)
        self.state = (survive | born).view(np.uint8)
        self.time_step += 1
```

Підрахунок сусідів реалізовано через `np.pad(mode='wrap')` — це забезпечує тороїдальне загортання без циклів. Операція повністю векторизована (NumPy), тому один крок Game of Life виконується за мікросекунди навіть для великих сіток.

2.2 Клас Agent

```
class Agent:
    def __init__(self, world, start_pos=(0, 0), perception_radius=1):
        self.world = world
        self.position = list(start_pos)
        self.perception_radius = perception_radius

    def perceive(self):
        r = self.perception_radius
        x, y = self.position
        ys = np.arange(y - r, y + r + 1) % self.world.height
        xs = np.arange(x - r, x + r + 1) % self.world.width
        return self.world.state[np.ix_(ys, xs)]
```

Метод `perceive()` використовує `np.ix_` для індексування з тороїдальним загортанням — елегантний NumPy-прийом, що замінює вкладені цикли.

2.3 Клас SaliencyMap

Карта салієнтності — механізм **привернення уваги** агента до «цікавих» областей сприйняття.

```
class SaliencyMap:
    def update(self, perception):
        # Motion saliency: ЕМА різниці між послідовними кадрами
        diff = np.abs(frame - self._prev_frame)
        self._motion_ema =  $\alpha$  * self._motion_ema + (1 -  $\alpha$ ) * diff
```

```
# Static saliency: Gaussian blur живих клітин
self._static_map = cv2.GaussianBlur(frame, (k, k),  $\sigma$ )
```

- **Motion saliency** (рухова): відстежує клітини, що змінюються між кадрами. Висока для осциляторів і рухомих об'єктів.
- **Static saliency** (статична): Гаусівський розмив живих клітин. Висока для скупчень живих клітин.

Метод `segment()` виконує **сегментацію** — виділяє зв'язні компоненти зі статичної карти, перетворюючи безперервну салієнтність на дискретні **блоби** (об'єкти-кандидати).

2.4 Клас WorldModel

Внутрішня модель світу агента. Зберігає растрову карту та координує трекінг об'єктів.

```
class WorldModel:
    def __init__(self, width, height, saliency=None):
        self._values = np.zeros((height, width), dtype=np.uint8) # растрова карта
        self._known = np.zeros((height, width), dtype=bool)      # маска спостереж
        self._tracker = ObjectTracker(width, height)              # трекер об'єктів

    def update(self, perception, agent_pos, perception_radius, world_time):
        # 1. Штапуємо сприйняття в карту
        self._values[idx] = perception
        self._known[idx] = True
        # 2. Оновлюємо салієнтність і виділяємо блоби
        self.saliency.update(perception)
        blobs = self.saliency.segment()
        # 3. Трекінг об'єктів
        self._tracker.update(blobs, agent_pos, r, world_time, self._values)
```

2.5 Клас ObjectTracker та WorldObject

Трекер підтримує множину об'єктів між кадрами. Кожен об'єкт описується структурою:

```
@dataclass
class WorldObject:
    id: int # унікальний ідентифікатор
    centroid: np.ndarray # (x, y) — позиція в світових координатах
    area: int # площа в клітинах
    bbox: tuple # (x, y, w, h) — обмежувальна рамка
    motion_score: float # середня рухова салієнтність
    first_seen: int # крок першого спостереження
    last_seen: int # крок останнього спостереження
    snapshot: np.ndarray | None # RGB-знімок поточного стану
    history: ObjectHistory | None # темпоральна історія кадрів
```

Алгоритм трекінгу (метод `update()`):

1. **Конвертація координат:** блоби з локальних координат патчу → світові координати
2. **Жадібне зіставлення:** кожен існуючий об'єкт шукає найближчий блоб (тороїдальна відстань $< match_radius$)
3. **Оновлення:** для зіставленого об'єкта — оновити центроїд, площу, записати кадр в історію
4. **Створення нових:** незіставлені блоби стають новими об'єктами (з перевіркою дублікатів)

Дедублікація: перед створенням нового об'єкта перевіряємо, чи немає поблизу (в радіусі `dedup_radius`) існуючого об'єкта із подібною площею. Це запобігає повторному створенню об'єкта, коли агент повертається до вже відвіданої зони.

Об'єкти ніколи не видаляються з пам'яті — навіть коли вони покидають зону сприйняття.

2.6 Клас ObjectHistory

Зберігає темпоральну послідовність бінарних кадрів об'єкта для **детекції періоду**.

```
@dataclass
class ObjectHistory:
    half_size: int                # половина розміру квадратного кропу
    frames: deque(maxlen=64)      # буфер останніх кадрів
    period: int | None = None     # виявлений період (1 = still, 2 = blinker, ...)
    cycle: list | None = None     # один повний цикл кадрів
```

Кожен кадр — це квадратний кроп з `_values`, прив'язаний до **центроїда** об'єкта (а не до його `bbox`). Це забезпечує просторову стабільність навіть при легких коливаннях обмежувальної рамки.

Детекція періоду (`_try_detect_period()`): для кожного кандидата-періоду $k = 1, 2, \dots$ перевіряє, чи кадри з відстанню k «однакові» (середня різниця $\leq \varepsilon$). Потрібно `MIN_CONFIRM = 3` підтвердження підряд.

2.7 Політики руху (action.py)

Паттерн **Стратегія**: базовий клас `Policy` з методом `__call__(agent, model) → direction`.

Політика	Стратегія
<code>GreedyExplorerPolicy</code>	Один крок: рухатись туди, де найбільше невідомих клітин
<code>NearestUnknownPolicy</code>	Глобально: навігація до найближчої невідомої клітини (тороїдальний Манхеттен)
<code>PatternSeekPolicy</code>	Фаза А: дослідити весь світ → Фаза В: знайти найподібніший об'єкт до зразка

2.8 Клас PatternMatcher

Ковзне вікно + метрика відстані для порівняння бінарного зразка з кожним об'єктом.

```
class PatternMatcher:
    def find_best_matches(self, model, top_k=5):
        for obj in model.objects.values():
            if obj.history.cycle is not None:
                # Фазово-інваріантне порівняння: мін. відстань по всіх фазах циклу
                score = min(self._score_object(f) for f in obj.history.cycle)
            else:
                # Fallback: порівняння з bbox-регіоном растрової карти
                score = self._score_object(region)
```

Підтримувані метрики:

- **Hamming:** $\sum |w_{ij} - p_{ij}|$ — кількість бітів, що відрізняються
- **Cosine:** $1 - \cos(w, p)$ — косинусна відстань між вирівняними векторами

2.9 Візуалізація (viz_rerun.py)

Використовує бібліотеку **Rerun** для інтерактивної інспекції всіх внутрішніх станів:

Панель	Зміст
World State	Повна сітка світу + позиція агента
Observation	Поточне вікно сприйняття
World Map	Внутрішня карта: сіре = невідомо, біле/чорне = відомо
Motion / Static Saliency	Теплові карти салиєнтності
Policy Log	Поточна політика, фаза, покриття
Objects	Обмежувальні рамки + знімки + інфо про період
Coverage % / Object Count	Часові ряди метрик

3 Структура коду

3.1 Точка входу

```
# 2d_binary_discrete_ca_saliency.py
import numpy as np
from pathlib import Path
from simulation import Simulation

np.random.seed(42)

if __name__ == "__main__":
    pattern_file = Path(__file__).parent / 'patterns' / 'arrow.json'
    Simulation(warmup=1000, pattern_file=str(pattern_file)).run()
```

3.2 Головний цикл (simulation.py)

```
class Simulation:
    def run(self):
        while not self._quit:
            # 1. Обробка клавіш (pynput → queue)
            # 2. Крок Game of Life: world.step()
            # 3. Оновлення моделі: model.update(agent.perceive(), ...)
            # 4. Автоматичний рух: policy(agent, model) → direction
            # 5. Візуалізація: rr.log_frame(...)
            time.sleep(0.05)
```

Управління клавіатурою:

Клавіша	Дія
w/s/a/d	Рух агента (ручний режим)
SPACE	Пауза / продовження
t	Перемикання ручний ↔ автоматичний режим
p	Зміна політики (в автоматичному режимі)
q / ESC	Вихід

3.3 Формат патернів

Патерни зберігаються як JSON-файли в директорії patterns/:

```
{
  "name": "Blinker",
  "pattern": [
    [0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0]
  ]
}
```

4 Завдання

4.1 Завдання 1 — Запуск та ознайомлення

Клонуйте репозиторій та запустіть симуляцію:

```
git clone https://github.com/hronoses/intro2agi_course
cd intro2agi_course
pip install -r requirements.txt
python practice_2/2d_binary_discrete_ca_saliency.py
```

Ознайомтесь з інтерфейсом Regun. Спробуйте:

- Ручне управління (w/s/a/d) — спостерігайте, як змінюється внутрішня карта

- Автоматичний режим (t) — подивіться, як агент досліджує світ
- Зміну політик (p) — порівняйте поведінку GreedyExplorer та NearestUnknown
- Паузу (SPACE) — дослідіть поточний стан об'єктів у вкладці Objects

4.2 Завдання 2 — Об'єктна модель світу

4.2.1 2.1. Групування об'єктів за ознаками

Реалізуйте функцію, яка знаходить **групи однакових об'єктів** серед усіх відстежених WorldObject.

Два об'єкти вважаються «однаковими», якщо:

1. Вони мають **однаковий період** (обидва still life, або обидва blinker з періодом 2, тощо)
2. Їх **цикли** (послідовності бінарних кадрів) подібні з точністю до фази. Використайте фазово-інваріантну відстань Геммінга:

$$d(A, B) = \min_{\phi \in [0, k)} \sum_{j=0}^{k-1} \|a_j - b_{(j+\phi) \bmod k}\|_1$$

Об'єкти з $d(A, B) \leq \theta$ (поріг подібності) належать до одного класу.

Підказка

Почніть з простого: спершу групуйте лише за періодом (без порівняння циклів). Потім додайте фазово-інваріантне порівняння. Використовуйте вже реалізований `_score_object()` з `PatternMatcher` як базу.

4.2.2 2.2. Побудова об'єктної моделі

На основі результату групування побудуйте **об'єктну модель** світу — колекцію пар (ознака, позиції):

```
def build_object_model(model) -> list[dict]:
    """
    Повертає список класів об'єктів:
    [
        {
            'feature': np.ndarray або list[np.ndarray], # патерн або цикл
            'period': int,                               # період
            'positions': [(x1, y1), (x2, y2), ...],      # центроїди всіх екземплярів
            'count': int                                 # кількість екземплярів
        },
        ...
    ]
    """
```

Для **статичних об'єктів** (period == 1): ознака — один бінарний кадр.

Для **осциляторів** (period > 1): ознака — список із k бінарних кадрів (цикл).

Що робити з об'єктами без виявленого циклу?

Деякі об'єкти можуть не мати виявленого періоду (наприклад, щойно з'явилися або хаотичні). Варіанти:

- **Ігнорувати** — включати в модель лише об'єкти з відомим періодом
- **Використовувати останній snapshot** — розглядати як still life за замовчуванням
- **Окрема категорія** period=None — «невідомий тип»

Оберіть підхід та обґрунтуйте у звіті.

4.3 Завдання 3 — Реконструкція та порівняння з реальністю

4.3.1 3.1. Реконструкція растрової карти з об'єктної моделі

Реалізуйте функцію, яка **відновлює** повну растрову карту з об'єктної моделі:

```
def reconstruct_from_objects(object_model: list[dict],
                             width: int, height: int) -> np.ndarray:
    """
    З об'єктної моделі відновити растрову карту WxH.

    Для кожного класу: взяти поточну фазу ознаки (feature)
    та «наштампувати» її на карту в усі позиції цього класу.
    Мертві клітини (фон) — нулі за замовчуванням.
    """
    grid = np.zeros((height, width), dtype=np.uint8)
    for cls in object_model:
        feature = cls['feature'] # для осциляторів — взяти потрібну фазу
        for (cx, cy) in cls['positions']:
            # наштампувати feature з центром у (cx, cy)
            ...
    return grid
```

4.3.2 3.2. Похибка реконструкції (відстань Геммінга)

Порівняйте реконструйовану карту з реальним станом середовища:

$$E_t = \frac{1}{W \cdot H} \sum_{x,y} |w_{x,y}^{(t)} - \hat{w}_{x,y}^{(t)}|$$

Це **нормалізована відстань Геммінга** — частка клітин, що відрізняються.

```
def hamming_error(real_world: np.ndarray, reconstructed: np.ndarray) -> float:
    return np.abs(real_world.astype(int) - reconstructed.astype(int)).mean()
```

Схема вимірювання

```
# Після того як агент дослідив увесь світ:
obj_model = build_object_model(simulation.model)
reconstructed = reconstruct_from_objects(obj_model, W, H)
error = hamming_error(simulation.world.state, reconstructed)
print(f"Hamming error: {error:.4f}")
```

Побудуйте графіки:

- E_t як функція часу (наскільки швидко застаріває об'єктна модель?)
- E для різних порогів групування θ

4.3.3 3.3. Порівняння розмірів моделей

Обчисліть та порівняйте обсяг пам'яті для трьох представлень:

Растрова карта (наївна):

$$|\mathcal{M}_{\text{raster}}| = W \times H \text{ бітів}$$

Об'єктна модель (без групування):

$$|\mathcal{M}_{\text{obj}}| = K \times (s_i^2 \times k_i + 2 \times \lceil \log_2 W \rceil) \text{ бітів}$$

де s_i^2 — розмір кроку об'єкта, k_i — його період, $2 \times \lceil \log_2 W \rceil$ — біти на координати позиції.

Об'єктна модель (з групуванням):

$$|\mathcal{M}_{\text{grouped}}| = \sum_{c=1}^C (s_c^2 \times k_c + n_c \times 2 \times \lceil \log_2 W \rceil) \text{ бітів}$$

де C — кількість класів, n_c — кількість екземплярів класу c . Ознака зберігається **один раз** на клас.

```
def model_size_bits(object_model, W, H):
    """Обчислити розмір об'єктної моделі в бітах."""
    coord_bits = 2 * int(np.ceil(np.log2(max(W, H))))
    total = 0
    for cls in object_model:
        feature = cls['feature']
        if isinstance(feature, list): # цикл
            feat_bits = sum(f.size for f in feature)
        else:
            feat_bits = feature.size
        total += feat_bits + cls['count'] * coord_bits
    return total
```

Заповніть таблицю та побудуйте графіки:

Модель	Розмір (біт)	Похибка E
Растрова карта	$W \times H$	0 (у зоні спостереження)
Об'єктна (без групування)	?	?
Об'єктна (з групуванням)	?	?

Компроміс розмір — точність

Об'єктна модель — це **стиснення з втратами**. Вона:

- Не зберігає фон (мертві клітини між об'єктами) — вважає його нульовим
- Може мати помилки сегментації (два близькі об'єкти зливаються або один розпадається)
- Не модулює хаотичні області, що не утворюють стійких паттернів

Чим менший поріг групування θ , тим точніша модель (більше класів), але й більший розмір.

4.4 Завдання 4 (Не обов'язкове) — Динамічні об'єкти

Основна проблема динамічного середовища: **об'єкти змінюються з часом**. Растрова карта застаріває миттєво для клітин поза сприйняттям. Як із цим справляється об'єктна модель?

4.4.1 4.1. Передбачення для осциляторів

Якщо об'єкт має відомий **період** k і ми знаємо його **цикл** $[f_0, f_1, \dots, f_{k-1}]$, ми можемо передбачити його стан в **будь-який майбутній момент**:

$$\hat{f}_i^{(t)} = f_{i, t \bmod k}$$

Реалізуйте передбачення стану осциляторів і включіть його у реконструкцію:

```
def reconstruct_with_prediction(object_model, width, height,
                                current_time, last_seen_times):
    """
    Для кожного об'єкта: якщо він має цикл, обчислити поточну фазу
    і використати відповідний кадр. Для still life – завжди той самий кадр.
    """
    grid = np.zeros((height, width), dtype=np.uint8)
    for cls in object_model:
        period = cls['period']
        for j, (cx, cy) in enumerate(cls['positions']):
            dt = current_time - last_seen_times[...]
            if period and period > 1:
                phase = dt % period
                frame = cls['feature'][phase]
            else:
                frame = cls['feature'] # або cls['feature'][0]
            # наштампувати frame з центром у (cx, cy)
```

```
...  
return grid
```

4.4.2 4.2. Вимірювання точності передбачення

Побудуйте графік E_t (похибка реконструкції) як функцію **часу після останнього спостереження** $\Delta t = t - t_{\text{last_seen}}$:

- Для **растрової карти**: E зростає з Δt (карта застаріває)
- Для **об'єктної моделі з передбаченням**: E має бути стабільною для осциляторів і still life

Що з рухомими об'єктами (spaceships)?

Рухомі об'єкти (наприклад, глайдер) зміщуються на $(\Delta x, \Delta y)$ за період. Об'єктна модель без урахування руху **швидко втрачає точність** для таких об'єктів, бо їх реальна позиція змінюється.

Варіанти розширення:

1. **Оцінка вектора швидкості**: відстежувати зміщення центроїда між кадрами та додати до моделі $v_i = (\Delta x/k, \Delta y/k)$. Передбачення: $\hat{p}_i^{(t)} = p_i^{(t_0)} + v_i \cdot (t - t_0)$
2. **Класифікація за рухливістю**: розділити об'єкти на «статичні» ($v \approx 0$) та «рухомі» ($v \neq 0$). Для статичних — лише цикл; для рухомих — цикл + швидкість.
3. **Прийняти втрату**: для навчальних цілей достатньо зафіксувати, що рухомі об'єкти створюють систематичну помилку, і **виключити їх** з моделі (або помітити як `type='unknown'`).

Реалізуйте хоча б один з цих варіантів та опишіть результати.

4.5 Завдання 5 — Аналіз та висновки

Підготуйте короткий звіт (1–2 сторінки), в якому:

1. **Подайте знайдені класи об'єктів**: для кожного класу — візуалізація ознаки (або циклу), період, кількість екземплярів, позиції.
2. **Порівняйте розміри моделей**: растрова vs об'єктна (з групуванням і без). При яких умовах об'єктна модель компактніша? Що відбувається при збільшенні розміру світу $W \times H$?
3. **Обговоріть компроміс**: точність реконструкції vs розмір моделі. Як поріг θ впливає на обидві величини?
4. **Передбачення**: наскільки точно об'єктна модель передбачає стан для still life, осциляторів і (якщо реалізовано) рухомих об'єктів?
5. **Рефлексія**: які типи «знань» про світ зберігає об'єктна модель, а які — ні? Що втрачається при стисненні?

5 Критерії оцінювання

#	Критерій	Бали
1	Запуск, ознайомлення з кодом (скріншоти Rerun)	5
2.1	Групування об'єктів за ознаками (фазово-інваріантне порівняння)	25
2.2	Побудова об'єктної моделі [feature, positions]	15
3.1	Реконструкція растрової карти з об'єктної моделі	15
3.2	Вимірювання похибки (Hamming distance) + графіки	10
3.3	Порівняння розмірів моделей (таблиця + графіки)	10
5	Звіт з аналізом та висновками	20
	Разом	100
4 □	Передбачення для динамічних об'єктів + аналіз (бонус)	+15

6 Корисні матеріали

- Базовий код: practice_2/ (усі модулі)
- Документація Rerun: rerun.io/docs
- Game of Life wiki: conwaylife.com/wiki
- Практика 1 (для порівняння підходів): practice_1/homework/instructions.qmd
- NumPy документація: numpy.org/doc